

Q33. Case Study: Employee Database Search using Binary Search

1. Aim

To implement the Binary Search algorithm in Python to efficiently search for an employee ID from a sorted employee database.

2. Problem Statement

A corporate system maintains employee records in sorted order for faster searching. Analyze how Binary Search improves retrieval speed, identify the requirements for Binary Search, apply the divide-and-conquer technique, and justify its time complexity.

3. Objectives

- Implement Binary Search in Python.
- Search an employee ID efficiently.
- Understand divide-and-conquer strategy.
- Compare Binary Search efficiency with Linear Search.

4. Software and Hardware Requirements

Software:

- Python 3.x
- Programiz Online Compiler / VS Code

Hardware:

- Windows PC/Laptop
- Minimum 4 GB RAM

5. Theory

Binary Search is an efficient searching algorithm that works only on sorted data. Instead of checking every element, it repeatedly divides the search space into two halves. At every iteration, the middle element is compared with the target value. If the target is smaller, the left half is searched; otherwise, the right half is searched. This divide-and-conquer strategy significantly reduces the number of comparisons.

6. Algorithm

1. Store employee IDs in sorted order.

2. Initialize low = 0 and high = last index.
3. Calculate middle index.
4. Compare middle element with target.
5. If equal, return index.
6. If target is greater, search right half.
7. If target is smaller, search left half.
8. Repeat until found or low > high.
9. If not found, display an appropriate message.

7. Python Source Code

```
def binary_search(arr, target):
    low = 0
    high = len(arr)-1
    while low <= high:
        mid = (low + high)//2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

employee_ids=[1001,1005,1010,1015,1020,1025,1030,1035,1040,1045]

print("Employee Database (Sorted)")
print(employee_ids)

target=int(input("Enter Employee ID to search: "))
index=binary_search(employee_ids,target)

if index!=-1:
    print("Employee ID",target,"found at index",index)
else:
    print("Employee ID",target,"not found.")
```

8. Input

Employee ID Entered: 1030

9. Output

Employee ID 1030 found at index 6.

10. Program Execution Screenshot

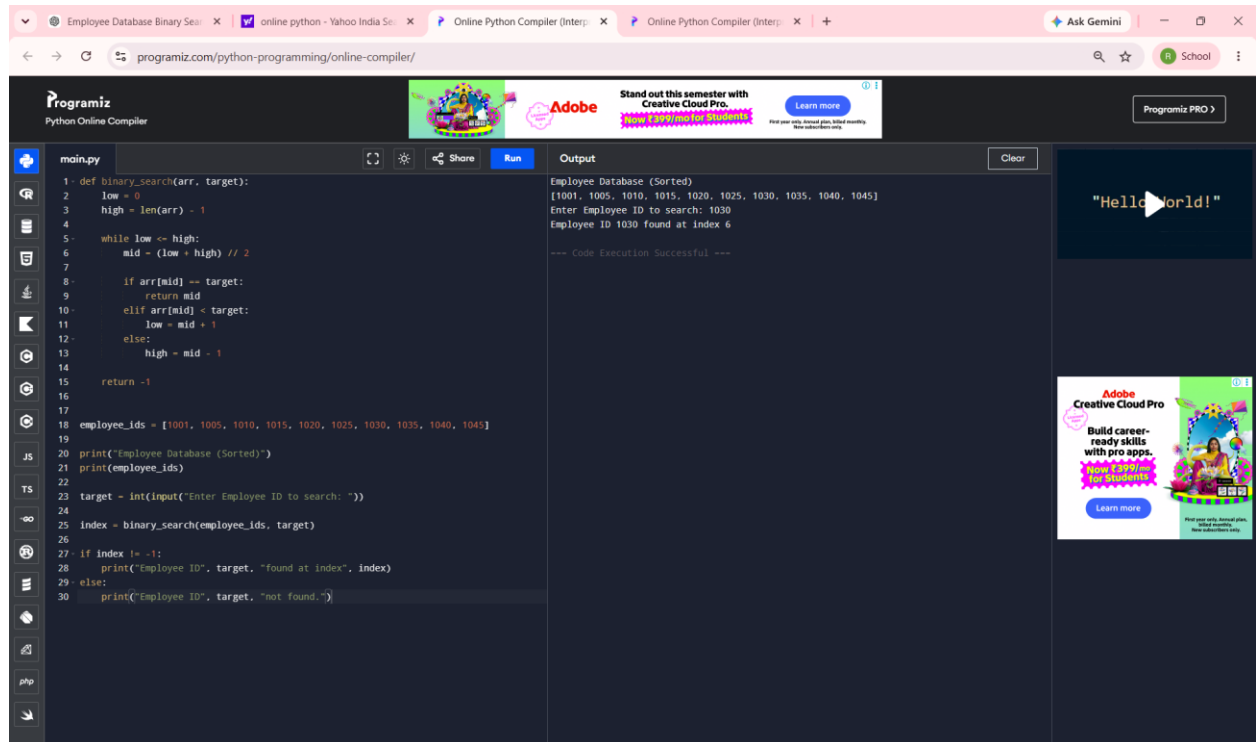


Figure: Successful execution of Binary Search program in Programiz.

11. Dry Run

Iteration	Low	High	Mid	Result
1	0	9	4	1020 < 1030, search right
2	5	9	7	1035 > 1030, search left
3	5	6	5	1025 < 1030, search right
4	6	6	6	1030 found

12. Time Complexity

Case	Complexity
Best Case	O(1)
Average Case	O(log n)
Worst Case	O(log n)

13. Space Complexity

O(1) for the iterative implementation.

14. Advantages

- Very fast for large sorted databases.
- Reduces search time significantly.
- Uses divide-and-conquer approach.
- Suitable for employee and student record systems.

15. Limitations

- Works only on sorted data.
- Sorting is required before searching if data is unsorted.

16. Applications

- Employee database systems
- Student record management
- Library management
- Banking systems
- Hospital record systems

17. Conclusion

Binary Search is one of the most efficient searching algorithms for sorted datasets. It reduces the search space by half in every iteration, giving a logarithmic time complexity of $O(\log n)$. Hence, it is highly suitable for fast retrieval of employee records in large corporate databases.